3.4 Controlling Association Scope

By default, associations look for objects only within the current module's scope. This can be important when you declare Active Record models within a module. For example:

```
module MyApplication
  module Business
    class Supplier < ActiveRecord::Base
       has_one :account
    end

    class Account < ActiveRecord::Base
       belongs_to :supplier
    end
  end
end
```

This will work fine, because both the `Supplier` and the `Account` class are defined within the same scope. But the following will *not* work, because `Supplier` and `Account` are defined in different scopes:

```
module MyApplication
  module Business
    class Supplier < ActiveRecord::Base
       has_one :account
    end
  end

  module Billing
    class Account < ActiveRecord::Base
       belongs_to :supplier
    end
  end
end
```

To associate a model with a model in a different namespace, you must specify the complete class name in your association declaration:

```
module MyApplication
  module Business
    class Supplier < ActiveRecord::Base
       has_one :account,
        class_name: "MyApplication::Billing::Account"
    end
  end

  module Billing
    class Account < ActiveRecord::Base
       belongs_to :supplier,
        class_name: "MyApplication::Business::Supplier"
    end
  end
end
```

3.5 Bi-directional Associations

It's normal for associations to work in two directions, requiring declaration on two different models:

```
class Customer < ActiveRecord::Base
  has_many :orders
end

class Order < ActiveRecord::Base
  belongs_to :customer
end
```